

Latency Minimization Resource Allocation and Trajectory Optimization for UAV-Assisted Cache-Computing Network With Energy Recharging

Peng Qin^{ID}, Member, IEEE, Xue Wu, Min Fu, Rui Ding^{ID}, and Yang Fu^{ID}

Abstract—Air-ground integrated network is able to make up for the limitations of small coverage as well as fixed resource deployment of ground 5G network and provide flexible services via edge computing. However, duplicated data may be offloaded to edge server, resulting in waste of network resources. In the meantime, due to the limited on-board energy storage of unmanned aerial vehicle (UAV), the endurance and trajectory optimization of UAV need to be focused on. Moreover, the high dynamics of network nodes will lead to the dilemma of information uncertainty and curse of dimensionality. In order to tackle the above challenges, we design a UAV-assisted heterogeneous cache-computing network model, among which, devices are divided into two categories: the request ground device (RGD) which requires task offloading and the free ground device (FGD) which can process offloaded tasks from the RGD through the device-to-device (D2D) link. Then we formulate a problem of jointly optimizing cache strategy, task segmentation, computing resource allocation and UAV trajectory planning to minimize the system latency constrained by UAV energy recharging. Due to the coexistence of discrete and continuous variables as well as coupling between long-term energy constraint and short-term decision making, we decompose it into two sub-problems. Low complexity matching algorithm is used to select the optimal UAV task caching strategy, while Lyapunov optimization is leveraged to decompose the second sub-problem, for which CVX is utilized to solve the task segmentation and local computing resource allocation, and soft actor-critic (SAC) approach is used to realize the computing resource allocation of FGD and UAV plus trajectory planning. Extensive simulation results showcase

the advantages of our algorithm in reducing the system latency compared to benchmarks.

Index Terms—UAV-assisted cache-computing network, energy recharging, resource optimization, soft actor-critic (SAC).

I. INTRODUCTION

WITH the fast development of communication and edge computing technologies, terminals have higher expectation for high-speed, reliable and low-latency communication services. However, because of the immanent limitations of network coverage, fixed resource deployment and poor emergency response ability of existing terrestrial 5G, it cannot provide seamless connection and on-demand computing services. For instance, ground base stations may be overloaded when handling massive access demands in hotspot areas, and in some application scenarios such as remote monitoring and emergency rescue, there is no available ground infrastructure. This necessitates paradigm shifts for fulfilling the hotspot enhancement and supplementary service requirements of terrestrial 5G network. Fortunately, the air-ground integrated network (AGIN) composed of unmanned aerial vehicle (UAV) and ground users is a better solution [1], [2], [3]. Among them, UAV can realize rapid deployment and flexible adjustment, provide wider communication coverage and assist terrestrial cellular network. Meanwhile, the cooperation between ground devices via device-to-device (D2D) link can further enhance the system performance [4]. However, for edge computing, duplicated data may be offloaded to the edge node, resulting in waste of network resources, i.e., the offloaded data for the recognizer process and feature library in Augmented Reality (AR) application. Therefore, it is urgent to combine cache with computing.

As a promising solution, caching computing tasks in UAV has attracted more and more attentions [5], [6], [7], in which UAV can be equipped with computing as well as cache resources to relieve the pressure on ground equipment. In fact, caching computing tasks on UAV in advance to provide low-latency services and reduce network congestion has become a new paradigm [8]. Specifically, for some tasks exhibit strong predictability or fixed timing patterns, their input data as well as required programs can be pre-cached at edge server to alleviate task uploading pressure [9]. Take virtual reality (VR) projection task as an example, after predicting

Received 5 November 2024; accepted 15 January 2025. Date of publication 27 January 2025; date of current version 18 August 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62201212, 62271201, in part by the Natural Science Foundation of Hebei Province under Grant F2022502017, in part by the Fundamental Research Funds for the Central Universities under Grant 2023JC003, in part by the National Key R&D Program of China under Grant 2023YFB2904700, and in part by the Baoding Fundamental Research Project under Grant 2272P003. The associate editor coordinating the review of this article and approving it for publication was G. Zheng. (Corresponding authors: Rui Ding; Yang Fu.)

Peng Qin, Min Fu, and Yang Fu are with the State Key Laboratory of Alternate Electrical Power System With Renewable Energy Sources, School of Electrical and Electronic Engineering, North China Electric Power University, Beijing 102206, China (e-mail: 120222201193@ncepu.edu.cn).

Xue Wu is with the State Key Laboratory of Alternate Electrical Power System With Renewable Energy Sources, School of Electrical and Electronic Engineering, North China Electric Power University, Beijing 102206, China, and also with China Mobile Communications Group Tianjin Company Ltd., Tianjin 300308, China.

Rui Ding is with China Satellite Network Group Company Ltd., Beijing 100029, China (e-mail: greatdn@qq.com).

Digital Object Identifier 10.1109/TCOMM.2025.3534587

the user's field of view (FoV), the edge server can cache popular 2D FoVs in advance, then directly project them into 3D content and deliver to VR devices [10]. Meanwhile, owing to the limited on-board energy storage of UAV, setting up charging stations to charge UAV and plan UAV's flight trajectory are essential. In addition, in actual AGIN, the channel state information changes in real time, resulting in the network falls into the dilemma of information uncertainty and curse of dimensionality. Therefore, how to realize reliable resource allocation with uncertain information has always been an open question. Furthermore, for delay-sensitive and computation-intensive complex tasks, such as video navigation and face detection, these tasks can typically be decomposed into multiple data-dependent subtasks. By implementing partial offloading to distribute these subtasks across multiple nodes for processing in parallel, the overall responsiveness of the system can be significantly enhanced [11], [12].

Since the complexity of traditional optimization method is usually high for dynamic environment, the deep reinforcement learning (DRL)-based approach has been widely discussed [13], [14], [15]. DRL is able to optimize the system performance via the prediction capabilities supported by deep learning and the decision-making capabilities provided by reinforcement learning (RL), which can deal with large state space and time-varying environment. However, for scenario with multiple agents, training and exploration will consume a lot of time and resource, resulting in slow convergence speed. Therefore, how to realize the cooperation between agents needs to be paid attention to.

To solve the above challenges, we design a UAV-assisted cache-computing network model, and present a problem of jointly optimizing cache strategy, task segmentation, computing resource allocation and UAV trajectory planning to minimize the system latency constrained by UAV energy recharging. Owing to the coexistence of discrete and continuous variables as well as coupling between long-term energy constraint and short-term decision making, Lyapunov optimization is leveraged to decompose it into two sub-problems, which can be solved one by one. The main contributions are summarized below.

- A UAV-assisted cache-computing network model is first designed, including multiple UAVs, the request ground device (RGD) which requires task offloading and the free ground device (FGD) which can process offloaded tasks from the RGD. As the air-based stations, UAVs are equipped with computing and cache resources. To take full advantage of terminal's resources, FGDs are allowed to cooperate with RGDs via D2D communication, which can alleviate the communication congestion especially in hot spots. Considering the practical difficulties of UAVs' limited energy storage, a charging station is included. Then we put forward the problem of jointly optimizing cache strategy, task segmentation, computing resource allocation and UAV trajectory planning to minimize the system latency.
- Due to the coexistence of discrete and continuous variables, it is a mixed-integer nonlinear programming (MINLP) problem. Therefore, we decompose it into two

TABLE I
KEY NOTATIONS

Notation	Description
$\mathcal{U} = \{U_1, U_2, \dots, U_M\}$	UAV set
$\mathcal{D}^R = \{D_1^R, D_2^R, \dots, D_N^R\}$	RGD set
$\mathcal{D}^F = \{D_1^F, D_2^F, \dots, D_K^F\}$	FGD set
$L = \{1, 2, \dots, L\}$	Time slot set
$c_{i,n,k}^F(l), c_{i,n,k}^U(l)$	Proportion of task offloaded to D^F/D^R
$\gamma_{n,m,l}^U(l), \gamma_{n,k}^F(l)$	Signal-to-noise-ratio (SNR) from D_n^R to U_m/D_k^F
$R_{n,m}^U(l), R_{n,k}^F(l)$	Uplink transmission rate from RGD to UAV/FGD
$k_{m,l}^U, \omega_{n,l}, \omega_{k,l}^F$	Coordinates of U_m, D_n^R, D_k^F
δ	Threshold
$Q_n^{com,loc}(l)$	Calculation buffer queue backlogs of D_n^R
$Q_n^{off,loc}(l)$	Offloading task buffer queue backlogs of D_n^R
$A_n^{com,loc}(l)$	Tasks received by D_n^R
$W_{n,m}^{off}(l), W_{n,k}^{off,F}(l)$	Tasks offloaded to U_m/D_k^F
$W_n^{loc}(l)$	Tasks processed by D_n^R
$W_n^{off}(l)$	Tasks offloaded from D_n^R
$T_n^{L,E}(l)$	Local processing latency
$T_{n,k}^{TR}(l), T_{n,k}^{U,TR}(l)$	Transmission delay from D_n^R to D_k^F/U_m
$Q_{n,k}^F(l), Q_{n,m}(l)$	Queue backlog of tasks offloaded from D_n^R stored at D_k^F/U_m
$G_n^{off,F}(l), G_{n,m}^{off}(l)$	Tasks executed by D_k^F/U_m
$E_{n,m}^{U,E}(t)$	Computing energy consumption of U_m
$E_m(l)$	UAV U_m propulsion energy consumption
c	Energy transfer rate from the charging station to U_m
$E_m^S(l)$	UAV battery energy state

subproblems: 1) UAV task cache strategy subproblem, and 2) task segmentation, resource allocation and UAV trajectory planning subproblem. For subproblem 1 with discrete optimization variable, the optimal UAV task cache strategy based on low complexity matching algorithm is proposed. For subproblem 2, Lyapunov optimization is used to transform the long-term issue into two short-term issues. CVX tool is applied to solve the convex problem of task segmentation and local computing resource allocation, while soft actor-critic (SAC) algorithm is applied to realize the computing resource allocation of FGDs and UAVs, as well as trajectory planning.

- Detailed theoretical analysis and extensive simulation experiments are carried out. The proposed approach is compared with the benchmark methods such as multi-agent deep Q network (MADQN), multi-agent deep deterministic policy gradient algorithm (MADDPG), SAC-trajectory algorithm (SAC-TR) and SAC-resource allocation algorithm (SAC-RA). Simulation results indicate that our method is superior to the benchmarks in terms of system latency.

For ease reference, the key notations are listed in Table I.

The article is arranged as follows. Section II summarizes the related work. Section III gives the air-ground cache-computing network model. Section IV describes the problem. Section V puts forward the UAV task cache strategy, task segmentation, resource allocation and UAV trajectory optimization algorithm. Section VI analyzes the simulation results, and Section VII is the conclusion.

II. RELATED WORKS

In recent years, air-ground network has attracted much attention, which integrates the advantages of aerial platform including UAV and high altitude platform station (HAPS) and ground infrastructure to enhance network coverage, capacity and reliability [16], [17], [18], [19], [20]. Literature [16] proposed an AGIN-power Internet of Things (AGIN-PIoT) network model, in which UAV was used to offer reliable and flexible edge computing services for PIoT devices. The terminal tasks were offloaded to UAVs, and UAVs chose the appropriate flight path to minimize the energy consumption. Literature [17] designed a UAV-assisted cluster non-orthogonal multiple access (C-NOMA) system, and UAV served as an air-based station for terminals. Then, a method of UAV trajectory planning and time slot allocation was presented. Literature [18] studied the UAV-assisted fog-enabled IoT network, in which UAV had computing power to provide edge services. Literature [19] proposed a C-NOMA system based on HAPS and UAVs in heterogeneous IoT, which achieved wide-area coverage and hotspot enhanced coverage. To sum up, UAV-assisted network offers a significant point of reference for this research.

Network resource optimization involves many aspects such as computing and caching [6], [21], [22], [24], [25], [26]. Literature [6] considered a cloud-based content distribution network. Based on the game of non-transferable utility alliance and Lyapunov optimization, content provider alliance, dynamic channel and the activity allocation strategy of UAVs were obtained. In paper [21], the combined caching, offloading and time allocation strategies were optimized through the alternating direction method of multipliers (ADMM) and penalty convex-concave procedure. Literature [22] proposed an optimization method of task allocation, terminal association and computing resource allocation based on the space-air-ground integrated architecture with wireless power transmission, in order to minimize the long-term average operating cost. In paper [24], considering the resource competition, a cache-enhanced computation offloading method was proposed. Literature [25] designed a heterogeneous network model for vehicle, which efficiently enhanced the energy efficiency and system spectrum efficiency via matching-based power control and subchannel allocation. However, most of the above studies only consider the single-dimensional resource optimization of cache or computing, which leads to poor overall system performance. Moreover, all above works adopt traditional optimization methods, which face fundamental challenges when dealing with large-scale terminals and complex environments such as the network dynamics.

As a cutting-edge technology in the area of artificial intelligence, machine learning has attracted much attention. In AGIN, the introduction of reinforcement learning (RL) provides new possibilities for the intelligent management of network resources [27], [28], [29], [30], [31]. Literature [27] designed a vehicular collaborative edge computing system model, and optimized task offloading and resource allocation via multi-armed bandits (MAB) method. Paper [28] proposed

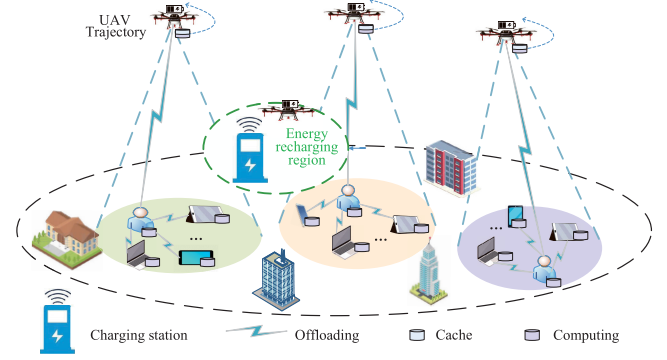


Fig. 1. UAV-assisted cache-computing network model.

a UAV-assisted wireless sensor network, which, based on Q-learning method, planned UAV trajectory by analyzing the UAV state and action, in order to collect data from sensor devices. Similarly, literature [29] proposed a flowlevel model-dependent deep RL (DRL) method to learn the best traffic-aware UAV trajectory. However, the goals of the above works are either to optimize the energy consumption, throughput or energy efficiency, and they all ignore considering the cache resource optimization which is essential especially in duplicated task offloading scenario.

III. SYSTEM MODEL

In Fig. 1, we design a UAV-assisted cache-computing network model, including multiple UAVs and ground devices. As the air-based station, UAV is equipped with computing and cache resources to provide users with communication and computing services. Let $\mathcal{U} = \{U_1, \dots, U_M\}$ represent UAV set. Devices are divided into two categories: the RGD which requires task offloading and the FGD which can process offloaded tasks from the RGD. Each group contains one RGD and multiple FGDs, and each FGD only belongs to one device group. Let $\mathcal{D}^R = \{D_1^R, D_2^R, \dots, D_N^R\}$ indicate N RGDs, $\mathcal{D}^F = \{D_1^F, D_2^F, \dots, D_K^F\}$ indicate K FGDs, respectively. Devices can communicate through D2D communication link, and the data transmission between UAV and RGD adopts orthogonal multiple access technology. Moreover, UAV has limited energy, so we set up a charging station at location $C^{loc} = (x^{loc}, y^{loc}, h^{loc})$, which can charge UAVs and prolong service time. The UAV can sense its energy state and fly to the charging area in time for charging.

Let the duration of flight be T seconds, and divide it into $\mathcal{L} = \{1, 2, \dots, L\}$ time slots, each length of which is Δt . In the l th time slot, the coordinate of UAV U_m is $k_{m,l}^U = (x_{m,l}^U, y_{m,l}^U, h_{m,l}^U)$, and the coordinate of D_n^R, D_k^F are $w_{n,l} = (r_{n,l}, s_{n,l}, 0)$ and $w_{k,l}^F = (r_{k,l}^F, s_{k,l}^F, 0)$. User D_n^R generates I_n^R computing tasks in T seconds, and $i \in \{1, 2, 3, \dots, I_n^R\}$ is the index of tasks on D_n^R . Let the size of computing task i on user D_n^R be $d_{i,n}$.¹ UAV can cache some computing tasks

¹In order to collect the necessary system information, the UAVs broadcast their locations to devices via downlink pilot channel, and the devices send the locations and task parameters to their visible UAVs, which then exchange information with other UAVs. Since the data size of the location and task information is small, we are able to collect and update the necessary information in the dynamic UAV network.

in advance to reduce the offloading pressure from users. Set $x_{n,m}^i$ as a binary variable, indicating whether the task i of D_n^R is cached on UAV U_m , i.e., if $x_{n,m}^i = 1$, it means that the task i is cached on U_m in advance; otherwise, $x_{n,m}^i = 0$. If the computing task generated by D_n^R in slot l has been cached on some UAV, it will be calculated directly, while if the computing task is not cached in advance, the user will offload it to the nearest UAV for reducing the response time. Furthermore, in order to prevent the UAV communication jam, it is allowed to offload the computing tasks to FGDs via D2D communication.

A. Communication Model

1) *UAV-Device Link*: In our model, the computation-intensive task I_n^R of D_n^R can be executed locally or offloaded to UAV or FGD. We consider partial offloading scheduling, where $c_{i,n,k}^F(l) \in [0, 1]$, $c_{i,n,m}^U(l) \in [0, 1]$ represent the proportion of task i of D_n^R offloaded to D_k^F and U_m , respectively. Therefore, $1 - c_{i,n,k}^F(l) - c_{i,n,m}^U(l) \in [0, 1]$ is the proportion of task i calculated locally. Specifically, the task size offloaded to the UAV U_m can be expressed as $c_{i,n,m}^U d_{i,n}$, while the task size offloaded to the ground terminal D_k^F can be represented as $c_{i,n,k}^F d_{i,n}$.

The mixed propagation model of line-of-sight(LoS) and non-line-of-sight (NLoS) communication is used to describe the transmission between UAVs and devices. LoS and NLoS loss (dB) from UAV U_m to D_n^R in slot l are respectively [32], [33]

$$l_{n,m,l}^{LoS} = l_{FS}(d_0) + 10\mu_{LoS} \log(d_{n,m,l}) + \psi_{\sigma LoS}, \quad (1)$$

$$l_{n,m,l}^{NLoS} = l_{FS}(d_0) + 10\mu_{NLoS} \log(d_{n,m,l}) + \psi_{\sigma NLoS}, \quad (2)$$

where μ_{LoS} and μ_{NLoS} are the large-scale path loss indices of LoS and NLoS links. $\psi_{\sigma LoS}$ and $\psi_{\sigma NLoS}$ are Gauss random variables with zero mean for LoS and NLoS links. $l_{FS}(d_0) = 20 \log(4\pi d_0 f_c / c)$ defines the free space path loss at the reference distance d_0 . f_c and c are the carrier frequency and the speed of light. $d_{n,m,l}$ is the distance between U_m and D_n^R in slot l , which is

$$d_{n,m,l} = \sqrt{h_{m,l}^U{}^2 + (x_{m,l}^U - r_{n,l})^2 + (y_{m,l}^U - s_{n,l})^2}. \quad (3)$$

LoS probability is expressed as

$$Pr(l_{n,m,l}^{LoS}) = \frac{1}{1 + X e^{-Y(\vartheta_{n,m,l} - X)}}, \quad (4)$$

where X and Y are environment-related parameters, and $\vartheta_{n,m,l} = \sin^{-1}(\frac{h_{m,l}^U}{d_{n,m,l}})$. Therefore, the average path loss of UAV-device (U2D) link is expressed as

$$\bar{l}_{n,m,l} = l_{n,m,l}^{LoS} \times Pr(l_{n,m,l}^{LoS}) + l_{n,m,l}^{NLoS} \times (1 - Pr(l_{n,m,l}^{LoS})). \quad (5)$$

The signal-to-noise ratio (SNR) of the U2D link from U_m to D_n^R is

$$\gamma_{n,m,l}^U = \frac{P_{n,m} |g_{n,m,l}|^2}{10 \bar{l}_{n,m,l} / 10 \sigma^2}, \quad (6)$$

where $P_{n,m}$ represents the UAV transmission power, σ^2 represents the power of additive white Gaussian noise, and $|g_{n,m,l}|^2$ is the small-scale fading gain.

Therefore, in slot l , the transmission rate between D_n^R and U_m is

$$R_{n,m}^U(l) = B_{n,m} \log_2(1 + \gamma_{n,m,l}^U), \quad (7)$$

where $B_{n,m}$ is the transmission bandwidth.

2) *Device-Device Link*: The distance from D_n^R to D_k^F is

$$d_{n,k}^F(l) = \sqrt{(r_{k,l}^F - r_{n,l})^2 + (s_{k,l}^F - s_{n,l})^2}. \quad (8)$$

In addition, in time slot l , if D_n^R offloads the task to D_k^F , the path loss from D_n^R to D_k^F is

$$h_{n,k}^F(l) = \frac{\beta_1 |g_{n,k,l}^F|^2}{d_{n,k}^F(l)^2}, \quad (9)$$

where β_1 represents the received power with a reference distance of 1 m between users, and $g_{n,k,l}^F$ characterizes the small-scale fading.

Orthogonal frequency division multiple access (OFDMA) scheme is adopted for the D2D communications, then the SNR between D_n^R and D_k^F is

$$\gamma_{n,k} = \frac{P_{n,k} h_{n,k}^F(l)}{\sigma^2}, \quad (10)$$

where $P_{n,k}$ represents the transmission power from D_n^R to D_k^F .

Let δ be the threshold indicating potential FGD candidates for each RGD, whose value should balance the offloading reliability and resource utilization. Thus, if $\gamma_{n,k} > \delta$, D_n^R and corresponding D_k^F will be included in the same device group. Each device group contains one RGD and multiple FGDs, and each FGD belongs to only one device group.

Then the transmission rate from D_n^R to D_k^F is

$$R_{n,k}(l) = B_{n,k} \log_2 \left(1 + \frac{P_{n,k} h_{n,k}^F(l)}{\sigma^2} \right), \quad (11)$$

where $B_{n,k}$ denotes the channel bandwidth.

B. Computing Model

UAVs and devices have limited buffer queues for caching tasks of D_n^R . $W_n^{loc}(l)$ and $W_n^{off}(l)$ are the size of tasks processed by local calculation and task offloading, respectively, which leave the corresponding buffer queue in slot l . The calculation and offloading task buffer queue backlogs of D_n^R are expressed as $Q_n^{com,loc}(l)$ and $Q_n^{off,loc}(l)$, i.e.,

$$\begin{aligned} Q_n^{com,loc}(l+1) &= \{Q_n^{com,loc}(l) - W_n^{loc}(l)\}^+ + A_n^{com,loc}(l), \end{aligned} \quad (12)$$

$$\begin{aligned} Q_n^{off,loc}(l+1) &= \{Q_n^{off,loc}(l) - W_n^{off}(l)\}^+ + A_n^{off,loc}(l), \end{aligned} \quad (13)$$

where $A_n^{com,loc}(l) = (1 - c_{i,n,k}^F(l) - c_{i,n,m}^U(l)) d_{i,n}$, $A_n^{off,loc}(l) = (c_{i,n,k}^F(l) + c_{i,n,m}^U(l)) d_{i,n}$, $\{\cdot\}^+ = \max(\cdot, 0)$.

The number of tasks offloaded to U_m and D_k^F is determined based on the offloading rate and the task backlog, and is respectively expressed as

$$W_{n,m}^{off}(l) = \min \{ Q_{n,m}^{off,loc}(l), \Delta t R_{n,m}^U(l) \}, \quad (14)$$

$$W_{n,k}^{off,F}(l) = \min \{ Q_{n,k}^{off,loc}(l), \Delta t R_{n,k}(l) \}. \quad (15)$$

Suppose that the computing resource of RGD in time slot l is $f_n^L(l)$ cycle/s, and the computing resources allotted by U_m and D_k^F to D_n^R are $f_{n,m}^U(l)$ cycle/s and $f_{n,k}^F(l)$ cycle/s. C represents the number of CPU cycles for processing 1-bit data. If the task of D_n^R is processed locally, we have

$$W_n^{loc}(l) = \min \left\{ Q_n^{com,loc}(l), \frac{\Delta t f_n^L(l)}{C} \right\}, \quad (16)$$

and the local processing latency is

$$T_n^{L,E}(l) = \min \left\{ \frac{Q_n^{com,loc}(l) C}{f_n^L(l)}, \Delta t \right\}. \quad (17)$$

If D_n^R offloads the task to D_k^F in time slot l , the transmission delay is

$$T_{n,k}^{TR}(l) = \min \left\{ \frac{Q_{n,k}^{off,loc}(l)}{R_{n,k}(l)}, \Delta t \right\}. \quad (18)$$

Suppose that in time slot l , the queue backlog of tasks offloaded from D_n^R stored at D_k^F is $Q_{n,k}^F(l)$, we have

$$Q_{n,k}^F(l+1) = \{ Q_{n,k}^F(l) - G_{n,k}^{off,F}(l) \}^+ + W_{n,k}^{off,F}(l), \quad (19)$$

where $G_{n,k}^{off,F}(l) = \min \left\{ Q_{n,k}^F(l), \frac{\Delta t f_{n,k}^F(l)}{C} \right\}$ represents the task executed by D_k^F , and $W_{n,k}^{off,F}(l)$ denotes the task size offloaded to D_k^F in the l th slot.

Then the task processing delay at D_k^F is

$$T_{n,k}^{L,E}(l) = \min \left\{ \frac{Q_{n,k}^F(l) C}{f_{n,k}^F(l)}, \Delta t \right\}. \quad (20)$$

Analogously, if D_n^R offloads task i to U_m in slot l , the task transmission delay between D_n^R and U_m is

$$T_{n,m}^{U,TR}(l) = \min \left\{ \frac{Q_{n,m}^{off,loc}(l)}{R_{n,m}^U(l)}, \Delta t \right\}. \quad (21)$$

Suppose that in time slot l , the queue backlog of tasks offloaded from D_n^R stored at U_m is $Q_{n,m}(l)$, we have

$$Q_{n,m}(l+1) = \{ Q_{n,m}(l) - G_{n,m}^{off}(l) \}^+ + W_{n,m}^{off}(l), \quad (22)$$

where $G_{n,m}^{off}(l) = \min \left\{ Q_{n,m}(l), \frac{\Delta t f_{n,m}^U(l)}{C} \right\}$ indicates the size of tasks executed by UAV, and $W_{n,m}^{off}(l)$ is the size of task offloaded to U_m .

Then the processing latency of U_m executing the task of D_n^R is

$$T_{n,m}^{U,E}(l) = \min \left\{ \frac{Q_{n,m}(l) C}{f_{n,m}^U(l)}, \Delta t \right\}. \quad (23)$$

In addition, the computing energy consumption of U_m is

$$E_{n,m}^{U,E}(l) = K_{n,m}^{U,E} (f_{n,m}^U(l))^3 \min \left\{ \frac{Q_{n,m}(l) C}{f_{n,m}^U(l)}, \Delta t \right\}, \quad (24)$$

where $K_{n,m}^{U,E}$ represents the power coefficient between D_n^R and U_m .

C. UAV Flight Energy Consumption Model

Define $v_{m,l}^U = [v_{m,l}^x, v_{m,l}^y, v_{m,l}^z]$ as the speed of UAV U_m in slot l , where $v_{m,l}^x, v_{m,l}^y, v_{m,l}^z$ are the velocity components of $v_{m,l}^U$ in the three-dimensional Cartesian coordinate system respectively, and they are constant during the time slot l . Based on the classic aircraft dynamics, the UAV propulsion energy consumption $E_m(l)$ consists of the blade shape power to overcome blade resistance and the power to overcome UAV gravity, which can be expressed as

$$E_m^P(l) = c_1 + c_2 \|v_{m,l}^U\|_2^2, \quad (25)$$

where $c_1 = \frac{1}{8} C_{D0} \mu s A \omega^3 r^3 + \kappa \frac{W_m^{3/2}}{\sqrt{2} \mu A}$, $c_2 = \frac{3 C_{D0}}{8 \omega^2 r^2} \mu s A \omega^3 r^3$. C_{D0} is the contour drag coefficient, which depends on the geometry of the rotor blade. $W_m = m_m g$ is the gravity of UAV m , m_m and g represent the UAV weight and gravity acceleration respectively. μ denotes the air density, s represents the rotor solidity, and A is the total area of UAV rotor disk. ω denotes blade angular velocity and r represents rotor radius.

When U_m enters the charging space ($D_m^{ch}(l) \leq D_0$), it will charge the battery with the energy charging rate² [33]

$$E_m(l) = c \mathbf{1}(D_m^{ch}(l) \leq D_0), \quad (26)$$

where $\mathbf{1}(\cdot)$ denotes the indicator function, $c \geq c_1 + c_2 v_{max}^2$ is the energy transfer rate from the charging station to U_m , and $D_m^{ch}(l)$ is the distance from U_m to the charging area. Due to the limited battery buffer, we have the upper and lower mapping processes $L_m(l)$ and $U_m(l)$, as well as the upper and lower energy queue boundary N_E and 0. The energy mapping process can be expressed as $\int_0^l \mathbf{1}_{(E_m^S(l) < 0)} dL_m(l) = 0$, $\int_0^l \mathbf{1}_{(E_m^S(l) > N_E)} dU_m(l) = 0$, where $E_m^S(l)$ denotes the UAV battery energy in slot l (Joule). Therefore, if the UAV energy buffer is null, it cannot consume energy, while if the UAV is full of energy, it cannot be additionally charged. In other words, the mapping process ensures that the length of the energy queue is bounded in the $[0, N_E]$ domain.

Then the UAV battery energy state is updated by

$$E_m^S(l) = E_m^S(l-1) - \sum_{n=1}^N E_{n,m}^{U,E}(l) + E_m(l) - E_m^P(l) + L_m(l) - U_m(l). \quad (27)$$

IV. PROBLEM FORMULATION

A. Problem Formulation

Our optimization goal is to minimize the average latency of all requested tasks in the network by collaborative design of UAV cache strategy, task segmentation, resource allocation

²This UAV charging model is reasonable to characterize the efficient power transfer mechanisms, e.g., laser. Besides, the proposed approach can adapt to more complex charging models, considering the impact of UAV angle and distance on charging rate, which will be investigated in future work.

and UAV trajectory. Since the size of the calculated results is usually far smaller, the feedback delay can be ignored [16]. Thus, the total latency $T_n(l)$ in time slot l consists of the time of D_n^R for local computing as well as task transmission and computing time to D_k^F or UAV, which is defined as:

$$T_n(l) = \left(T_n^{L,E}(l) + \sum_{k=1}^K \left(T_{n,k}^{TR}(l) + T_{n,k}^{L,E}(l) \right) + \sum_{m=1}^M \left(T_{n,m}^{U,TR}(l) + T_{n,m}^{U,E}(l) \right) \right). \quad (28)$$

Then the average latency is

$$T = \frac{1}{L} \sum_{l=1}^L \sum_{n=1}^N \sum_{i=1}^{I_n^R} \times \left((1 - x_{n,m}^i) T_n(l) + \sum_{m=1}^M x_{n,m}^i T_{n,m}^{U,E}(l) \right). \quad (29)$$

Consequently, the optimization problem is formulated as

P0 :

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{C}, \mathbf{F}, \mathbf{Q}} \quad & T \\ \text{s.t.} \quad & C1 : x_{n,m}^i \in \{0, 1\}, \\ & C2 : \sum_{n=1}^N \sum_{i=1}^{I_n^R} x_{n,m}^i \leq C_m, \\ & C3 : c_{i,n,k}^F(l) \in [0, 1], \\ & C4 : c_{i,n,m}^U(l) \in [0, 1], \\ & C5 : f_n^L(l) \leq f_{n,max}, \\ & C6 : \sum_{n=1}^N f_{n,k}^F(l) \leq f_{k,max}, \\ & C7 : \sum_{n=1}^N f_{n,m}^U(l) \leq f_{m,max}, \\ & C8 : x_{min} \leq x_{m,l}^U \leq x_{max}, \\ & C9 : y_{min} \leq y_{m,l}^U \leq y_{max}, \\ & C10 : h_{min} \leq h_{m,l}^U \leq h_{max}, \\ & C11 : v_{m,l}^U \leq v_{max}, \\ & C12 : E_m^S(s) - \sum_{i=s}^l \left(\sum_{n=1}^N E_{n,m}^{U,E}(i) - E_m(i) + E_m^P(i) \right) \\ & \quad - U_m(l) \\ & \quad \geq \frac{c_1 + c_2 v_{max}^2}{v_{max}} \left(\|k_{m,s}^U + \sum_{i=s}^l v_{m,i}^U - C^{loc}\|_2 - D_0 \right), \\ & C13 : \|k_{m,l}^U - k_{j,l}^U\| \geq D_{min}, \forall m, j, m \neq j, \\ & C14 : \bar{Q}_n^{com,loc}(l) < \infty, \bar{Q}_n^{off,loc}(l) < \infty, \\ & \quad \bar{Q}_{n,k}^F(l) < \infty, \bar{Q}_{n,m}(l) < \infty, \end{aligned}$$

where $X = \{x_{n,m}^i, \forall i \in I_n^R, n \in \mathcal{D}^R, k \in \mathcal{D}^F\}$ is the set of caching strategy for UAV to cache user computing tasks, $C = \{c_{i,n,k}^F(l), c_{i,n,m}^U(l), \forall i \in I_n^R, n \in \mathcal{D}^R, k \in \mathcal{D}^F, m \in \mathcal{U}, l \in \mathcal{L}\}$ represents the segmentation of RGD computing tasks, $F =$

$\{f_n^L(l), f_{n,k}^F(l), f_{n,m}^U(l), \forall n \in \mathcal{D}^R, k \in \mathcal{D}^F, m \in \mathcal{U}, l \in \mathcal{L}\}$ denotes the computing resource allocation, and $\mathbf{Q} = \{k_{m,l}^U, \forall m \in \mathcal{U}, l \in \mathcal{L}\}$ is the UAV flight trajectory. $C1 - C2$ indicate that the caching strategy of UAV is a binary variable, and C_m indicates the maximum UAV cache capacity. $C3 - C4$ represent the segmentation ratio of computing tasks on UAV and FGD, respectively. $C5 - C7$ are the computing resource constraints of users and UAVs. $C8 - C10$ indicate that UAV flies in a limited area. $C11$ represents the speed constraint of UAV flight, and v_{max} represents the maximum flight speed. $C12$ represents the power and trajectory control of UAV, which ensures that UAV has enough energy to fly to the charging place. $C13$ is the collision constraint between UAVs, and D_{min} is the minimum safe distance between UAVs. $C14$ ensures network stability.

We divide the problem into two subproblems: the optimization subproblem of UAV task cache strategy, as well as the optimization subproblem of task segmentation, resource allocation and UAV trajectory. Among them, the first one can be expressed as

$$\begin{aligned} \mathbf{P1} : \min_{\mathbf{x}} \quad & T \\ \text{s.t.} \quad & C1, C2, \end{aligned} \quad (30)$$

and the second one is denoted by

$$\begin{aligned} \mathbf{P2} : \min_{\mathbf{C}, \mathbf{F}, \mathbf{Q}} \quad & T \\ \text{s.t.} \quad & C3 - C14. \end{aligned} \quad (31)$$

Note that due to **P2** involves multiple variables as well as coupling between long-term queue constraints and short-term decision making, it is still NP-hard. Thus, we use Lyapunov optimization to split and transform it into a series of deterministic subproblems.

B. Lyapunov Optimization

The most significant component of Lyapunov optimization is the optimal control of the dynamic system using the Lyapunov function, so as to achieve the network stability. Assuming that the network state in slot l is $S(l) = \{Q_n^{com,loc}(l), Q_n^{off,loc}(l), Q_{n,k}^F(l), Q_{n,m}(l)\}$. Lyapunov function can be defined as

$$L(S(l)) = \frac{1}{2} \left\{ \sum_{n=1}^N \left(Q_n^{com,loc}(l)^2 + Q_n^{off,loc}(l)^2 + \sum_{k=1}^K Q_{n,k}^F(l) + \sum_{m=1}^M Q_{n,m}(l) \right) \right\}. \quad (32)$$

Then Lyapunov drift is defined as the expectation of Lyapunov function changing in adjacent time slots:

$$\Delta L(S(l)) = \mathbb{E}[L(S(l+1)) - L(S(l)) \mid S(l)]. \quad (33)$$

In order to minimize the system delay while ensuring the stability of the queue, Lyapunov drift plus penalty is utilized, which is denoted as Lyapunov drift plus average delay expectation multiplied by the weight parameter V :

$$\Delta_V L(S(l)) = \Delta L(S(l)) + V \mathbb{E}[\varphi(l) \mid S(l)]. \quad (34)$$

$\varphi(l) = \sum_{n=1}^N \sum_i^{I_n^R} \left((1 - x_{n,m}^i) T_n(l) + \sum_{m=1}^M x_{n,m}^i T_{n,m}^{U,E}(l) \right)$ represents the system latency in the time slot l . V is non-negative, and it makes a trade-off between penalty minimization and queue stability.

Theorem 1: While $V \geq 0$, given the network state, it is further deduced that the upper bound of $\Delta_V L(S(l))$ is

$$\begin{aligned} \Delta_V L(S(l)) &\leq \sum_{n=1}^N \mathbb{E} [Q_n^{com,loc}(l) (A_n^{com,loc}(l) - W_n^{loc}(l))] \\ &+ \sum_{n=1}^N \mathbb{E} [Q_n^{off,loc}(l) (A_n^{off,loc}(l) - W_n^{off}(l))] \\ &+ \sum_{n=1}^N \sum_{k=1}^K \mathbb{E} [Q_{n,k}^F(l) (W_{n,k}^{off,F}(l) - G_{n,k}^{off,F}(l))] \\ &+ \sum_{n=1}^N \sum_{m=1}^M \mathbb{E} [Q_{n,m}(l) (W_{n,m}^{off}(l) - G_{n,m}^{off}(l))] \\ &+ V \mathbb{E}[\varphi(l) | S(l)] + const, \end{aligned} \quad (35)$$

where $const$ is a constant.

Proof: See Appendix A. $\blacksquare$

Without considering the changes of random variables within slots, the key point is to optimize $\Delta_V L(S(l))$ in each time slot. According to Lyapunov theory, the optimization problem of task segmentation, resource allocation and UAV trajectory is decomposed into two subproblems, each of which can be handled independently.

1) *Task segmentation and local computing resource allocation:*

SP1 :

$$\begin{aligned} \min_{c_{i,n,k}^F(l), c_{i,n,m}^U(l), f_n^L(l)} \Gamma(l) &= \sum_{n=1}^N [Q_n^{com,loc}(l) \\ &\times \left(A_n^{com,loc}(l) - \frac{\Delta t f_n^L(l)}{C} \right) \\ &+ Q_n^{off,loc}(l) A_n^{off,loc}(l)] \\ &+ V \sum_{n=1}^N \sum_i^{I_n^R} \\ &\left((1 - x_{n,m}^i) \min \left\{ \frac{Q_n^{com,loc}(l) C}{f_n^L(l)}, \Delta t \right\} \right) \\ s.t. \quad &C3, C4, C5, \\ &C15 : \frac{\Delta t f_n^L(l)}{C} \leq Q_n^{com,loc}(l). \end{aligned} \quad (36)$$

2) *Computing resource allocation of FGD and UAV as well as UAV trajectory optimization:*

SP2 :

$$\begin{aligned} \min_{f_{n,k}^F(l), f_{n,m}^U(l), \mathbf{Q}} \Psi(l) &= \sum_{n=1}^N \left[-Q_n^{off,loc}(l) W_n^{off}(l) \right. \\ &+ \sum_{k=1}^K \left[Q_{n,k}^F(l) \left(W_{n,k}^{off,F}(l) - \frac{\Delta t f_{n,k}^F(l)}{C} \right) \right] \end{aligned}$$

$$\begin{aligned} &+ \sum_{m=1}^M \left[Q_{n,m}(l) \left(W_{n,m}^{off}(l) - \frac{\Delta t f_{n,m}^U(l)}{C} \right) \right] \\ &+ V \sum_{n=1}^N \sum_i^{I_n^R} \left((1 - x_{n,m}^i) \left(\sum_{k=1}^K (T_{n,k}^{TR}(l) + T_{n,k}^{L,E}(l)) \right. \right. \\ &\left. \left. + \sum_{m=1}^M (T_{n,m}^{U,TR}(l) + T_{n,m}^{U,E}(l)) \right) + \sum_{m=1}^M x_{n,m}^i T_{n,m}^{U,E}(l) \right) \\ s.t. \quad &C6, C7, C8, C9, C10, C11, C12, C13, \\ &C16 : \frac{\Delta t f_{n,k}^F(l)}{C} \leq Q_{n,k}^F(l), \\ &C17 : \frac{\Delta t f_{n,m}^U(l)}{C} \leq Q_{n,m}(l). \end{aligned} \quad (37)$$

V. JOINT CACHE STRATEGY, TASK SEGMENTATION, COMPUTING RESOURCE ALLOCATION AND UAV TRAJECTORY OPTIMIZATION FOR CACHE-COMPUTING AGIN

In this section, the solution for the above subproblems is proposed. Firstly, since **P1** is an optimization problem about binary variables, we adopt a low-complexity matching method to solve it. Then note that **SP1** is a convex subproblem, it can be solved by CVX. Due to incomplete global information, the traditional method cannot effectively cope with **SP2**. Therefore, it is expressed as a Markov decision process (MDP) and tackled by SAC. By solving these three subproblems, the final optimization result is obtained.

A. Matching-Based Task Caching Strategy

To find the easy-to-handle method for **P1**, the subproblem is expressed as a matching problem. Matching partner is UAV and user task set. **P1** can be solved by matching UAV and user tasks, with the goal of minimizing system delay.

For U_m and task i_n^R , construct a many-to-one matching Π , where $\Pi(m) = \{ \{i_n^R\} \mid m \in \Pi(i_n^R), |\Pi(m)| \leq 1, \forall m \}$, and $\{ |\Pi(i_n^R)| \leq C_m, \forall m, i_n^R \}$ represent the task of RGD and the maximum connection number of each UAV.

Then, the utility function of UAVs and tasks under matching Π is defined. The utility function of UAV U_m caching all user tasks is expressed as

$$\Omega_m(i_n^R) = \frac{1}{L} \sum_{l=1}^L \sum_{n=1}^N \sum_{i_n^R=1}^{I_n^R} (T_{n,m}^{U,E}(l)). \quad (38)$$

Similarly, i_n^R only cares about the delay of the UAV that caches i_n^R . Consequently, the utility function of task i_n^R under matching Π is expressed as

$$\Omega_{i_n^R}(m) = \frac{1}{L} \sum_{l=1}^L x_{n,m}^i T_{n,m}^{U,E}(l). \quad (39)$$

To describe the preference of matching partners, the preference relation between UAVs and i_n^R is defined. Owing to the intensive tasks of users, the preference list of UAVs is influenced by the matching outcomes of other UAVs, thus the preference relationship between UAV and task changes through the matching process. The matching stability of cache

³<https://pan.baidu.com/s/1O3Eti7QuZbL7c8ldwGWonQ?pwd=icn2>

strategy subproblem cannot be ensured. Accordingly, the definition of blocking pair is proposed. Through avoiding the occurrence of blocking pairs, the stable matching between tasks and UAVs is realized.

Definition 1: For a pair of UAVs $(U_m, U_{m'})$, when $i_n^R \in \Pi(m) \& \bar{i}_n^R \in \Pi(m')$, if the exchange matching $\Pi_{m'}^m$ does not reduce the utility of all participants and at least one utility increases, then it can be called a blocking pair matching Π . If the blocking pair of Π does not exist, then matching Π achieves stability.

Through the above definition, we present an optimization algorithm of UAV cache strategy to obtain a stable matching result, formulated in **Algorithm 1**. It starts from computing task i_n^R , and chooses the most preferred U_m from its preference list. While the selected UAV U_m is vacant, the matching pair $(i_n^R, U_{m'})$ is directly added to Π . On the other hand, when there is no vacancy in U_m , that is, $|\Pi(m)| == C_m$, the computing task i_n^R will be compared with all the computing tasks currently matching UAV U_m . If the computing task i_n^R is better than the worst matching computing task $\bar{i}_n^R$, the computing task i_n^R will be exchanged with the computing task $\bar{i}_n^R$. When the priority lists of all computing tasks and UAVs are empty or not blocked, **Algorithm 1** terminates, and computing tasks and UAVs achieve stable matching.

B. Task Segmentation and Local Computing Resource Allocation

From (36), we derive the second derivative of the objective function of **SP1** as

$$\begin{aligned}
 & \frac{\partial^2 \Gamma(l)}{\partial c_{i,n,k}^F(l)^2} \\
 &= \frac{\partial^2 \Gamma(l)}{\partial c_{i,n,k}^F(l) \partial c_{i,n,m}^U(l)} = \frac{\partial^2 \Gamma(l)}{\partial c_{i,n,k}^F(l) \partial f_n^L(l)} \\
 &= \frac{\partial^2 \Gamma(l)}{\partial c_{i,n,m}^U(l) \partial c_{i,n,k}^F(l)} = \frac{\partial^2 \Gamma(l)}{\partial c_{i,n,m}^U(l)^2} \\
 &= \frac{\partial^2 \Gamma(l)}{\partial c_{i,n,m}^U(l) \partial f_n^L(l)} \\
 &= \frac{\partial^2 \Gamma(l)}{\partial f_n^L(l) \partial c_{i,n,k}^F(l)} = \frac{\partial^2 \Gamma(l)}{\partial f_n^L(l) \partial c_{i,n,m}^U(l)} = 0, \quad (40) \\
 & \frac{\partial^2 \Gamma(l)}{\partial f_n^L(l)^2} \\
 &= V \sum_i^{I_n^R} \left((1 - x_{n,m}^i) \min \left\{ \frac{2Q_{n,m}^{com,loc}(l)C}{f_n^L(l)^3}, 0 \right\} \right) > 0. \quad (41)
 \end{aligned}$$

Because the Hessian matrix of $\Gamma(l)$ is a semi-positive definite matrix, whilst all constraints are convex, **SP1** is a convex optimization problem with inequality constraints, thus we can use CVX toolbox to solve it.

C. Computing Resource Allocation of FGD and UAV as Well as UAV Trajectory Optimization

In our network model, FGD and UAV serve as two types of agents, which determine computing resource allocation and

Algorithm 1 Matching-Based Computing Task Cache Algorithm (MCTC)

```

1: Initialize: Preference lists  $PL_m^U, PL_i^C$  for UAVs and
   computing tasks
2: Set the matching  $\Pi = \emptyset, op = 1$ 
3: while  $op == 1$  do
4:   Set  $op = 0$ 
5:   for each UAV  $m$  do
6:     if  $PL_m^U \neq \emptyset$  then
7:       Select the computing task  $i_n^R$  in the preference list
          $PL_m^U$  as  $\Pi(m)$ 
8:       if the number of participants of  $PL_m^U ==$  maxi-
         mum capacity of the computing tasks then
9:         Select the worst matching UAV  $m'$  in  $PL_m^U(i_n^R)$ 
10:        if  $m$  is better than  $PL_{m'}^U(m')$  then
11:          Swap matching  $m$  and  $m'$  in  $\Pi(i_n^R)$ 
12:        else
13:           $PL_m^U = PL_m^U \setminus i_n^R$ 
14:        end if
15:        Set  $op = 1$ 
16:      end if
17:       $\Pi(m) = \Pi(m) \cup (m, i_n^R)$ 
18:    end if
19:  end for
20: end while
21: Output matching results  $\Pi$ .

```

UAV locations to obtain the minimum system delay. Due to the dynamic change of the environment, the previous state and the previous action together trigger the environment to enter a new state. Under this circumstance, the optimization problem is expressed as an MDP $(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{Q})$. $\mathcal{S}$ denotes the state set of all agents, $\mathcal{A}$ denotes the action space of agents, $\mathcal{R}$ denotes the reward function, and $\mathcal{Q} \in [0, 1]$ represents the discount coefficient.

1) $\mathcal{S}$: According to the optimization problem, the state includes the network typology and task queue backlogs. Considering the partial observation of agents, FGD D_k^F and UAV U_m are able to obtain the information below

$$o_k^F(l) = \{k_{m,l}^U, Q_{n,m}(l)\}, \quad (42)$$

$$o_m^U(l) = \{k_{m,l}^U, Q_{n,k}^{off,loc}(l), Q_{n,k}^F(l), E_m^S(l)\}, \quad (43)$$

and the combination of all agents' observations forms state S .

2) $\mathcal{A}$: In this paper, we need to determine the movement of UAV (horizontal flight distance $L_m^U(l)$, horizontal angle $\theta_m(l)$ and vertical flight distance $\Delta h_m(l)$), the computing resource allocation of UAV $f_{n,m}^U(l)$, and the computing resource allocation of FGD $f_{n,k}^F(l)$. The action space of UAV is $a_m^U(l) = \{L_m^U(l), \theta_m(l), \Delta h_m(l), f_{n,m}^U(l)\}$ and that of FGD is $a_k^F(l) = \{f_{n,k}^F(l)\}$.

3) $\mathcal{R}$: While satisfying some constraints, the agents cooperate to minimize $\Psi(l)$. If all the constraints are satisfied, the reward function of the agent $r(l) = r_\iota(l), \forall \iota \in \mathcal{U} \cup \mathcal{D}^F$ is defined as the negative value of $\Psi(l)$. Otherwise, if some

Algorithm 2 SAC-Based Computing Resource Allocation of Free Users and UAVs and UAV Trajectory Optimization Algorithm (SAC-CRAT)

- 1: **Input:** Number of UAVs M , number of users N and free users K , UAV initialized position $k_{m,l}^U$, learning rate τ .
 - 2: **Output:** UAV's position $k_{m,l}^U$, computing free user's resource allocation $f_{n,k}^F(l)$ and UAV's resource allocation $f_{n,m}^U(l)$.
 - 3: **Initialize** actor network π_l , critic network Q_l with weights θ_l, β_l , target actor and critic network: $\pi_{\bar{\theta}_l}$ and $Q_{\bar{\beta}_l}$ with weights $\bar{\theta}_l \leftarrow \theta_l, \bar{\beta}_l \leftarrow \beta_l$.
 - 4: **for** each step of episode **do**
 - 5: Invoke task caching strategy in **Algorithm 1**.
 - 6: Obtain optimal task segmentation and local computing resource allocation by solving **SP1**.
 - 7: Initialize a random process for action exploration.
 - 8: each agent ι randomly chooses a_l
 - 9: Execute action $\mathbf{a} = \{a_1, a_2, \dots, a_{M+K}\}$, receive the reward $\mathcal{R}$ and observe the state S' .
 - 10: Store (S, a, r, S') into its replay buffer D
 - 11: **if** it's time for an update **then**
 - 12: Sample a random minibatch of S samples from D
 - 13: Update Q-function parameters with (48): $\beta_l \leftarrow \beta_l - \tau \nabla_{\beta_l} J_Q(\beta_l)$
 - 14: Update Q-function parameters with (49): $\theta_l \leftarrow \theta_l - \tau \nabla_{\theta_l} J_{\pi_l}(\theta_l)$
 - 15: Update actor's and critic's target networks' parameters according to (50): $\bar{\theta}_l = \kappa \theta_l + (1 - \kappa) \bar{\theta}_l, \bar{\beta}_l = \kappa \beta_l + (1 - \kappa) \bar{\beta}_l$
 - 16: **end if**
 - 17: **end for**
-

constraints are not met, there is corresponding punishment in the reward function $r(l)$. Therefore, owing to the above considerations, $r(l)$ can be expressed as

$$r(l) = \begin{cases} -\Psi(l), & \text{satisfy constraints} \\ -e_1 - e_2 - e_3 - e_4 - e_5[N - \sum_{l=0}^M N_l(l)], & \text{other} \end{cases} \quad (44)$$

where e_1, e_2, e_3, e_4 and e_5 respectively represent the penalties related to computing resource constraint, collision constraint, speed constraint, energy constraint and coverage constraint.

To solve **SP2**, we propose an optimization algorithm based on SAC. SAC is different from the traditional algorithm which aims at maximizing the agent's reward, and instead maximizes the reward as well as entropy of agent's actions at the same time. Entropy maximization encourages the agents to explore more diverse actions, which improves learning stability as well as policy generalization. The optimization algorithm is expressed in **Algorithm 2**. Firstly, the parameters are initialized randomly. In each step, agent collects state information and executes actions. After executing the action, the agent collects the reward and stores (S, a, r, S') in the replay buffer, then updates deep neural networks (DNN)

parameters. We iterate the process and stop training when the moving average of the return rate is stable at a constant value.

Let the agent's actor network be $\pi = \{\pi_1, \dots, \pi_{M+K}\}$, and the parameter of each actor network be $\theta = \{\theta_1, \dots, \theta_{M+K}\}$. π_{θ_l} is used to represent the actor network π_l with the parameter θ_l , which generates action the current state through the current observation o_l . Furthermore, Q_{β_l} is used to represent the critic network with the parameter β_l . The input of the critic network is the observation and action of all agents at present, and the output is Q-value Q_l .

In the training process, update actor network of agent ι needs to minimize the target $J_{\pi_l}(\theta_l)$:

$$J_{\pi_l}(\theta_l) = E[\zeta \log(\pi_{\theta_l}(\tilde{a}_l | o_l)) - Q_{\beta_l}(S, \tilde{a})], \quad (45)$$

where ζ is used to control whether to focus more on entropy or reward. Each agent utilizes a separate critic network $Q_{\beta_l}(S, \tilde{a})$, $\tilde{a} \sim \pi_{\theta_l}(\cdot | o_l)$, and the output is the Q value of action $\tilde{a}_l$. The actor network $\pi_{\theta_l}(\cdot | o_l)$ can get the probability distribution of the action only by using its own observation o_l . The agent can get the specific action $\tilde{a}_l$ through sampling the obtained probability distribution, and the differential formulation of the target can also be got through a multi-parameter sampling approach:

$$J_{\pi_l}(\theta_l) = E_{S \sim D, \tilde{a} \sim \pi_{\theta}}[\zeta \log(\pi_{\theta_l}(\tilde{a}_l | o_l)) - Q_{\beta_l}(S, \tilde{a})], \quad (46)$$

where the expectation in $J_{\pi_l}(\theta_l)$ can be sampled from the experience replay buffer D and approximately calculated, and the actor network is updated utilizing the random gradient descent method.

According to the maximum entropy principle, the critic network parameter β_l of the agent is updated through minimizing the Berman error $J_Q(\beta_l)$:

$$J_Q(\beta_l) = E_{(S, a, r, S') \sim D} \left[\frac{1}{2} (Q_{\beta_l}(S, a) - \chi)^2 \right], \quad (47)$$

where

$$\chi = r_l + \mathcal{G} E_{a' \sim \pi_{\theta}} [Q_{\bar{\beta}_l}(S', a') - \zeta \log(\pi_{\bar{\theta}_l}(a'_l | o'_l))], \quad (48)$$

$\bar{\beta}_l$ is the parameter of the agent's target critic network $Q_{\bar{\beta}_l}$, and $\bar{\theta}_l$ is the agent ι target actor network $\pi_{\bar{\theta}_l}$. Random sampling is performed in the experience replay buffer to obtain a fixed batch of samples for approximate calculation of $J_Q(\beta_l)$, in which the agent ι inputs the observation of the next state o'_l into the target actor network $\pi_{\bar{\theta}_l}$, then the acquired action probability distribution is sampled to obtain the agent's next action a'_l .

At the same time, after each parameter update of actor network and critic network, the target actor and target critic network need to be softly updated:

$$\begin{aligned} \bar{\theta}_l &= \kappa \theta_l + (1 - \kappa) \bar{\theta}_l, \\ \bar{\beta}_l &= \kappa \beta_l + (1 - \kappa) \bar{\beta}_l, \end{aligned} \quad (49)$$

where κ is the hyperparameter to control the soft update of the target network.

D. Property Analysis

In this section, the convergence, complexity and optimality of the algorithms are analyzed.

1) *Convergence*: For **Algorithm 1**, the size of UAV preference list as well as computing task preference list is limited, and according to the definition of matching, the target value decreases continuously in limited exchange operations until a stable matching result is obtained. Moreover, because **SP1** is a convex problem and the number of UAVs is confined, a convergent UAV task caching strategy can be found through CVX tool. For **Algorithm 2**, SAC algorithm is to train agents to get the maximum cumulative reward in an episode. In the process of training, the increase of cumulative reward means the improvement of strategy. When the strategy reaches the local optimum, the cumulative reward gradually converges.

2) *Complexity*: Firstly, suppose **Algorithm 1** needs I iterations to converge. N_c represents the total number of tasks that all users need to calculate, and the maximum number of exchange matches per iteration needs MN_c operations. Consequently, the complexity of **Algorithm 1** is $O(IMN_c)$. **SP1** is a convex subproblem, which optimizes task segmentation and local computing resource allocation through l_1 iterations, and its complexity is $O(l_1N_c(K+M) + l_1N)$. For **Algorithm 2** based on SAC, because the training can be done offline, only the implementation complexity is considered. Define J and I as the number of layers in the actor network and the critic network, respectively. Let J_j and I_i represent the number of neurons in the j layer and the i layer, respectively. Then, the training complexity is given by $O(ep_{\max}L(K+M)(\sum_{j=2}^{J-1}(J_{j-1}J_j + J_jJ_{j+1}) + \sum_{i=2}^{I-1}(I_{i-1}I_i + I_iI_{i+1})))$, where $ep_{\max}$ is the number of episodes, and L is the total number of time slots. Such complexity increases linearly with the number of FGDs and UAVs. As a result, the total computational complexity is $O(IMN_c + ep_{\max}L[(K+M)(\sum_{j=2}^{J-1}(J_{j-1}J_j + J_jJ_{j+1}) + \sum_{i=2}^{I-1}(I_{i-1}I_i + I_iI_{i+1})) + l_1N_c(K+M) + l_1N])$.

3) *Optimality*: **Algorithm 1** optimizes the UAV task cache strategy based on matching game. Through the exchange operation, the utility of all participants does not reduce, and at least one utility rises. Consequently, after each operation, the delay of the system is reduced, and an approximately optimal task cache result is gained. **SP1** is a convex subproblem, thus the global optimal method can be gained. **Algorithm 2** optimizes UAV trajectory, UAV and FGD's computing resources based on SAC, and ensures that the strategy is random enough by using entropy regularization. Increasing entropy will lead to more exploration, protecting the strategy from premature convergence to a local optimum, and finally obtaining near-optimal results. Moreover, the decoupling of **P1** and **P2** may induce performance degradation, whereas separately solving **SP1** and **SP2** derived from Lyapunov optimization does not compromise the optimality of **P2**, thus the whole solution leads to the sub-optimal solution of **P0**.

VI. SIMULATION RESULTS

This section verifies the performance of our approach through a large number of simulation experiments. Users are

TABLE II
SIMULATION PARAMETERS

Parameters	Values
h_{min}, h_{max}	60 m, 80 m
M	1-5
N	10-70
K	40-80
C	1000 CPU cycles/bit
V	10^8
$P_{n,m}$	30 dBm
$P_{n,k}$	23 dBm
$B_{n,m}$	10 MHz
$B_{n,k}$	20 MHz
φ	0.95
X, Y	11.9, 0.13
v_{max}	20 m/s
σ^2	-174 dBm/Hz
N_E	37000 Joule
c_1	272.6
c_2	0.0337
δ	35 dB
Discount factor	0.99
Learning rate	0.0003
ep_{max}	100
e_1, e_2, e_3, e_4, e_5	30, 20, 20, 50, 30

distributed independently in an area of 500 m $\times$ 500 m. The UAV maximum speed v_{max} is 20 m/s. Besides, the actor and critic networks of each SAC agent both have two fully connected hidden layers with 256 and 128 neurons. Other default parameter values are shown in Table I [3], [16], [17].

Our method is compared with the following benchmarks:

- 1) *Multi-agent deep deterministic policy gradient algorithm (MADDPG)*: MADDPG-based method is used for **SP2**, and other solution is the same with ours.
- 2) *Multi-agent deep Q network method (MADQN)*: MADQN-based method is used for **SP2**, and other solution is the same with ours.
- 3) *SAC-trajectory method (SAC-TR)*: An optimization method of task caching, task segmentation and computing resource allocation without trajectory design.
- 4) *SAC-resource allocation method (SAC-RA)*: An optimization method of task caching, task segmentation and trajectory design without computing resource allocation.

Fig. 2(a) shows the trajectories of 3 UAVs with 100 randomly distributed users for better explanation. It can be observed that the proposed learning scheme does not violate the energy security constraints, and jointly optimizes the resources and trajectories of UAVs according to the energy state of each UAV and the state of user request queue. As shown in Fig. 2(b), the UAV adaptively controls its flight altitude. Specifically, when the UAV passes through an area with low user density, it raises the altitude to expand the coverage and enhance LoS propagation, allowing more users to offload tasks and reduce queue backlogs; otherwise, the UAV flies close to users by decreasing the altitude, which improves the channel power gain and boosts the offloading performance of high-density user area. Moreover, the remaining energy of UAV over time is shown in Fig. 2(c). One can observe that the energy is continuously consumed during the service time. Particularly, when the energy level is low, the UAV progressively approaches the altitude lower boundary, preparing to move closer to the charging station.

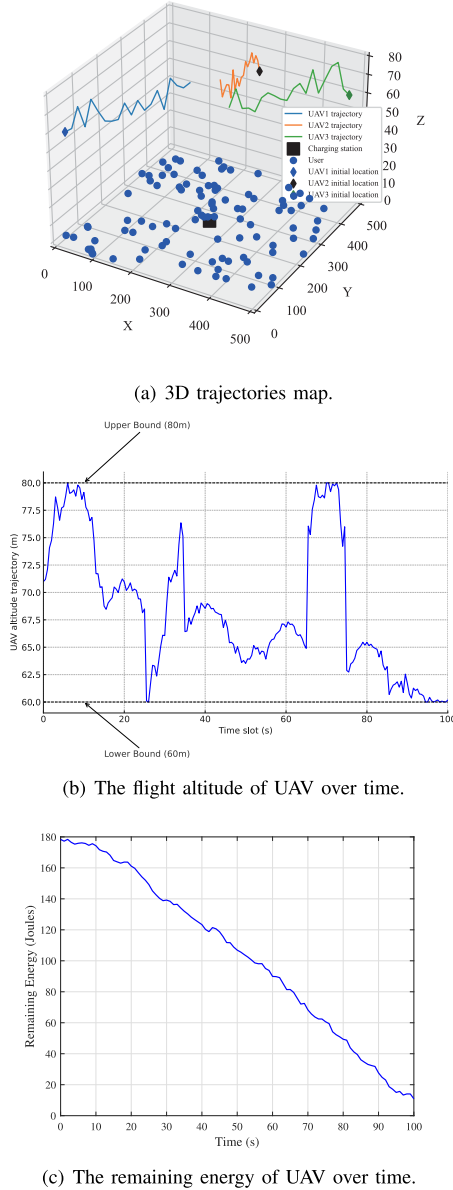


Fig. 2. The trajectories and remaining energy of UAVs.

Fig. 3 showcases the average latency for different methods. With the increase of users, the system average delay increases. This is due to the increase of users, resulting in an increase in the amount of tasks that need to be processed, therefore increasing the delay. Besides, when the number of users is relatively small, the increase in delay is quite slow. This is because the resources at UAVs and FGDs are sufficient and redundant to respond to the generated tasks, hence adding more users does not significantly affect the delay performance. As seen in the figure, our algorithm performs best, because compared with MADDPG algorithm, it can better explore the unknown state, with higher learning efficiency. Also note that, MADQN algorithm deals with the optimization of discrete space, SAC-TR algorithm does not consider the UAV trajectory plan, and SAC-RA ignores the reasonable allocation of computing resources, which prove the effectiveness of our approach.

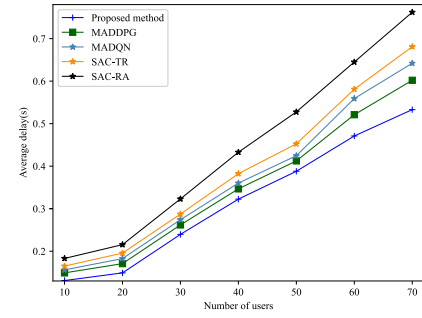


Fig. 3. Average latency versus number of users.

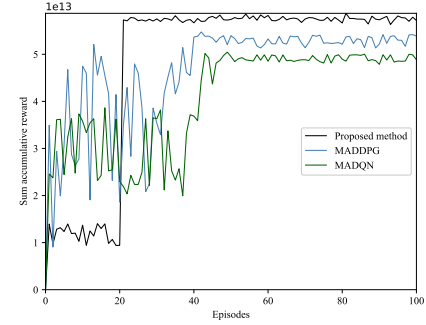


Fig. 4. Sum accumulative reward.

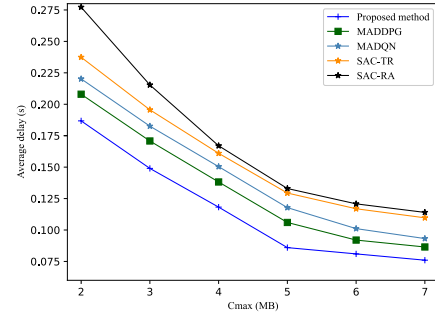
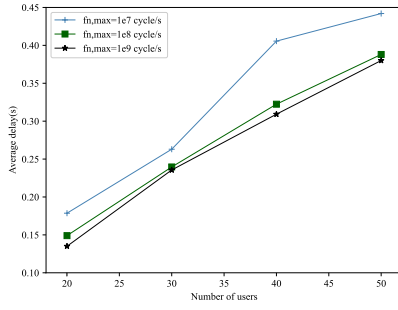


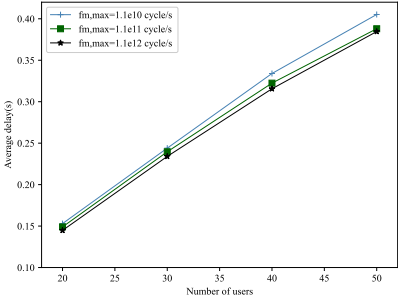
Fig. 5. Relationship between cache capacity and delay.

Fig. 4 illustrates the total cumulative reward of all agents. As seen in the figure, the performance of our algorithm is superior and it converges faster. To be specific, at the beginning of the training phase, the curve increases with the fluctuation. This is because users and UAVs learn the computing resource allocation and the optimization of UAV trajectory, and agents explore the cooperation to provide transmission services to reduce the delay. The proposed method converges when all agents learn the appropriate actions. Compared with MADDPG algorithm, the improvement of our algorithm in the performance is 12.5%, and the convergence is better. After training the SAC framework in offline, we can load the well-trained actor networks to yield real-time decisions in online execution stage, which consumes negligible time.

Fig. 5 gives the relationship between cache capacity and latency. As seen in the figure, with the increase of cache capacity, the average delay decreases. This is because the increasing cache capacity makes the computing tasks of UAV cache more, reduces the delay of processing computing tasks and saves the time of task offloading. However, once the cache capacity



(a) User computing power.



(b) UAV computing power.

Fig. 6. Influence of maximum computing power.

exceeds a certain threshold, the rate of latency reduction slows down, primarily because the cache resources have reached saturation. At this point, the constraint of other resources (e.g., CPU frequency and bandwidth) becomes the bottleneck of the further decrease in latency. Also seen in the figure, the delay of our algorithm is lower, and the performance is superior. Compared with MADDPG and MADQN algorithms, the delay is reduced by 16.9% and 27.2%, respectively.

Fig. 6 shows the influence of the maximum computing power of users and UAVs. As seen in the figure, with the increase of the number of users, the average delay is also increasing. Fig. 6(a) shows the influence of users' maximum computing power on performance. With the improvement of computing power, the efficiency of users' computing resources is improved, and the time delay of processing tasks is reduced by cooperating with UAVs. Fig. 6(b) shows the influence of the maximum UAV computing power on the average delay. Similarly, the improvement of UAV's computing power accelerates the task processing speed and reduces the network pressure and time delay.

The influence of weight parameter V on the system is illustrated in Fig. 7. With the increase of V , the average delay decreases, while the average queue backlog is rising progressively, causing the network stability to deteriorate. Note that, the larger V is, the lower the average delay is. However, the average queue backlog is also increasing, which makes it difficult to guarantee the stability of the system, thus revealing the trade-off between the optimal operating cost of the network and the stability. Consequently, a reasonable V value should be selected to control the system without causing network instability.

Fig. 8 and Fig. 9 display the average latency for different number of UAVs and FGDs, respectively. As seen in the

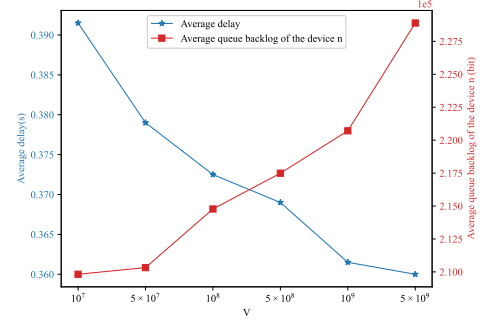
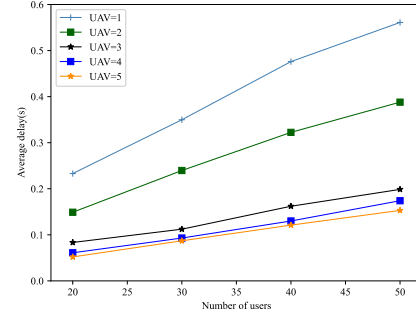
Fig. 7. The influence of V on the system.

Fig. 8. Average latency for different number of UAVs.

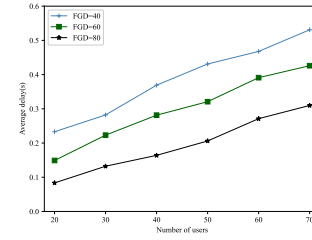


Fig. 9. Average latency for different number of FGDs.

Fig. 8, with the growing up of users' number, the latency also increases. This is owing to the incorporation of more UAVs enhances the opportunities for task offloading and edge computation, while simultaneously reducing the task load and queue backlog for each individual UAV, resulting in a decrease in overall latency. Fig. 9 shows the impact of different numbers of FGDs on the average latency. It can be clearly seen that, for a given number of RGDs, increasing the number of FGDs significantly re-duces the average latency. This is because more FGDs are available to handle tasks, in-creasing the likelihood of selecting an FGD with better channel conditions, thereby im-proving task processing efficiency and reducing latency. Furthermore, as the number of RGDs increases, resource competition intensifies, leading to an increase in the average system latency.

VII. CONCLUSION

In this paper, a UAV-assisted cache-computing network model was designed. Considering the practical difficulty of UAV's limited energy storage, a charging station was utilized to charge UAV, and the system latency minimization problem of UAV's task cache strategy optimization, task segmentation, resource allocation and UAV trajectory plan was put forward.

This problem involved the joint optimization of multi-domain resources, thus was decomposed into several subproblems, which were solved separately. The simulation results demonstrated the advantage of our algorithm in reducing the delay. In later work, we will take the satellite into consideration to further improve network performance.

REFERENCES

- [1] P. Qin, Y. Fu, X. Zhao, K. Wu, J. Liu, and M. Wang, "Optimal task offloading and resource allocation for C-NOMA heterogeneous air-ground integrated power Internet of Things networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 11, pp. 9276–9292, Nov. 2022.
- [2] S. Zhang, W. Quan, J. Li, W. Shi, P. Yang, and X. Shen, "Air-ground integrated vehicular network slicing with content pushing and caching," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 9, pp. 2114–2127, Sep. 2018.
- [3] P. Qin et al., "Joint trajectory plan and resource allocation for UAV-enabled C-NOMA in air-ground integrated 6G heterogeneous network," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3421–3434, Nov. 2023.
- [4] Z. Wang, J. Guo, Z. Chen, L. Yu, Y. Wang, and H. Rao, "Robust secure UAV relay-assisted cognitive communications with resource allocation and cooperative jamming," *J. Commun. Netw.*, vol. 24, no. 2, pp. 139–153, Apr. 2022.
- [5] M. Chen, W. Saad, and C. Yin, "Liquid state machine learning for resource and cache management in LTE-U unmanned aerial vehicle (UAV) networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 3, pp. 1504–1517, Mar. 2019.
- [6] A. Asheralieva and D. Niyato, "Game theory and Lyapunov optimization for cloud-based content delivery networks with device-to-device and UAV-enabled caching," *IEEE Trans. Veh. Technol.*, vol. 68, no. 10, pp. 10094–10110, Oct. 2019.
- [7] G. Zheng, C. Xu, M. Wen, and X. Zhao, "Service caching based aerial cooperative computing and resource allocation in multi-UAV enabled MEC systems," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 10934–10947, Oct. 2022.
- [8] N. Zhao et al., "Caching UAV assisted secure transmission in hyperdense networks based on interference alignment," *IEEE Trans. Commun.*, vol. 66, no. 5, pp. 2281–2294, May 2018.
- [9] P. Qin, M. Fu, Y. Fu, R. Ding, and X. Zhao, "Collaborative edge computing and program caching with routing plan in C-NOMA-enabled space-air-ground network," *IEEE Trans. Wireless Commun.*, vol. 23, no. 12, pp. 18302–18315, Dec. 2024.
- [10] Z. Liu, N. Garg, and T. Ratnarajah, "Multi-agent federated reinforcement learning strategy for mobile virtual reality delivery networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 1, pp. 100–114, Jan. 2024.
- [11] H. Guo, X. Zhou, J. Wang, J. Liu, and A. Benslimane, "Intelligent task offloading and resource allocation in digital twin based aerial computing networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 10, pp. 3095–3110, Oct. 2023.
- [12] H. Guo, Y. Wang, J. Liu, and C. Liu, "Multi-UAV cooperative task offloading and resource allocation in 5G advanced and beyond," *IEEE Trans. Wireless Commun.*, vol. 23, no. 1, pp. 347–359, Jan. 2024.
- [13] N. H. Chu, D. T. Hoang, D. N. Nguyen, N. Van Huynh, and E. Dutkiewicz, "Joint speed control and energy replenishment optimization for UAV-assisted IoT data collection with deep reinforcement transfer learning," *IEEE Internet Things J.*, vol. 10, no. 7, pp. 5778–5793, Apr. 2023.
- [14] O. S. Oubbati, M. Atiquzzaman, A. Baz, H. Alhakami, and J. Ben-Othman, "Dispatch of UAVs for urban vehicular networks: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 13174–13189, Dec. 2021.
- [15] P. Qin, Y. Wang, Z. Cai, J. Liu, J. Li, and X. Zhao, "MADRL-based URLLC-aware task offloading for air-ground vehicular cooperative computing network," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 6716–6729, Jul. 2024.
- [16] P. Qin, Y. Fu, Y. Xie, K. Wu, X. Zhang, and X. Zhao, "Multi-agent learning-based optimal task offloading and UAV trajectory planning for AGIN-power IoT," *IEEE Trans. Commun.*, vol. 71, no. 7, pp. 4005–4017, Jul. 2023.
- [17] Z. Na, Y. Liu, J. Shi, C. Liu, and Z. Gao, "UAV-supported clustered NOMA for 6G-enabled Internet of Things: Trajectory planning and resource allocation," *IEEE Internet Things J.*, vol. 8, no. 20, pp. 15041–15048, Oct. 2021.
- [18] X. Huang, X. Yang, Q. Chen, and J. Zhang, "Task offloading optimization for UAV-assisted fog-enabled Internet of Things networks," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 1082–1094, Jan. 2022.
- [19] P. Qin, Y. Zhu, X. Zhao, X. Feng, J. Liu, and Z. Zhou, "Joint 3D-location planning and resource allocation for XAPS-enabled C-NOMA in 6G heterogeneous Internet of Things," *IEEE Trans. Veh. Technol.*, vol. 70, no. 10, pp. 10594–10609, Oct. 2021.
- [20] J. Wang, C. Jiang, Z. Wei, C. Pan, H. Zhang, and Y. Ren, "Joint UAV hovering altitude and power control for space-air-ground IoT networks," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1741–1753, Apr. 2019.
- [21] W. Wen, Y. Cui, T. Q. S. Quek, F. Zheng, and S. Jin, "Joint optimal software caching, computation offloading and communications resource allocation for mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7879–7894, Jul. 2020.
- [22] J. Liu, X. Zhao, P. Qin, S. Geng, and S. Meng, "Joint dynamic task offloading and resource scheduling for WPT enabled space-air-ground power Internet of Things," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 2, pp. 660–677, Mar. 2022.
- [23] Y. Dong, S. Guo, Q. Wang, S. Yu, and Y. Yang, "Content caching-enhanced computation offloading in mobile edge service networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 1, pp. 872–886, Jan. 2022.
- [24] P. Qin, Y. Fu, X. Feng, X. Zhao, S. Wang, and Z. Zhou, "Energy-efficient resource allocation for parked-cars-based cellular-V2V heterogeneous networks," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 3046–3061, Feb. 2022.
- [25] S. Yu, R. Langar, X. Fu, L. Wang, and Z. Han, "Computation offloading with data caching enhancement for mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 67, no. 11, pp. 11098–11112, Nov. 2018.
- [26] P. Qin, Y. Fu, G. Tang, X. Zhao, and S. Geng, "Learning based energy efficient task offloading for vehicular collaborative edge computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 8, pp. 8398–8413, Aug. 2022.
- [27] S. Fu et al., "Energy-efficient UAV-enabled data collection via wireless charging: A reinforcement learning approach," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 10209–10219, Jun. 2021.
- [28] V. Saxena, J. Jalden, and H. Klessig, "Optimal UAV base station trajectories using flow-level models for reinforcement learning," *IEEE Trans. Cogn. Commun. Netw.*, vol. 5, no. 4, pp. 1101–1112, Dec. 2019.
- [29] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and L. Hanzo, "Multi-agent deep reinforcement learning-based trajectory planning for multi-UAV assisted mobile edge computing," *IEEE Trans. Cognit. Commun. Netw.*, vol. 7, no. 1, pp. 73–84, Mar. 2021.
- [30] R. Liu, Y. Ma, X. Zhang, and Y. Gao, "Deep learning-based spectrum sensing in space-air-ground integrated networks," *J. Commun. Inf. Netw.*, vol. 6, no. 1, pp. 82–90, Mar. 2021.
- [31] Y.-J. Chen, K.-M. Liao, M.-L. Ku, F. P. Tso, and G.-Y. Chen, "Multi-agent reinforcement learning based 3D trajectory design in aerial-terrestrial wireless caching networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 8201–8215, Aug. 2021.
- [32] M. Chen, M. Mozaffari, W. Saad, C. Yin, M. Debbah, and C. S. Hong, "Caching in the sky: Proactive deployment of cache-enabled unmanned aerial vehicles for optimized quality-of-experience," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 5, pp. 1046–1061, May 2017.
- [33] S. Chai and V. K. N. Lau, "Multi-UAV trajectory and power optimization for cached UAV wireless networks with energy and content recharging-demand driven deep learning approach," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 10, pp. 3208–3224, Oct. 2021.



Peng Qin (Member, IEEE) received the B.S. and Ph.D. degrees in information and communication engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2009 and 2014, respectively. From 2012 to 2013, he was a Visiting Scholar with the University of Victoria, Victoria, BC, Canada. He is currently an Associate Professor with the School of Electrical and Electronic Engineering, North China Electric Power University. His research interests include resource allocation in the Internet of Things, smart grid communications, space air ground integrated networks, and vehicular networks. He was a recipient of the International Communications Signal Processing and Systems Conference Best Paper Award, and the International Conference on Artificial Intelligence in China Best Paper Award in 2019, 2020, 2021, and 2022.



Xue Wu received the M.Sc. degree from the School of Electrical and Electronic Engineering, North China Electric Power University, in 2024. She is currently an Engineer with China Mobile Communications Group Tianjin Company Ltd. Her research interests include the Internet of Things, space air ground integrated networks, and wireless communication network resource optimization.



Rui Ding received the B.S. and Ph.D. degrees in electronics and telecommunications from Southeast University, Nanjing, China, in 2001 and 2011, respectively. He is currently a Professor with China Satellite Network Group Company Ltd., Beijing, China. His research interests include satellite communication and integrated satellite-terrestrial communications networks.



Min Fu received the B.S. degree from North China Electric Power University in 2022, where she is currently pursuing the M.Sc. degree with the School of Electrical and Electronic Engineering. Her research interests include space air ground integrated networks and wireless communication network resource optimization.



Yang Fu received the B.S. degree from North China Electric Power University in 2022, where he is currently pursuing the Ph.D. degree with the School of Electrical and Electronic Engineering. His research interests include resource allocation in smart grid communications, space air ground integrated networks, vehicular networks, and the Internet of Things.